

1.1 Die Warteschlange (Queue)

In der Arztpraxis wird in der Reihenfolge der Ankunft aufgerufen. Das **Projekt Praxis Website: Station 1.1** verwaltet das Wartezimmer zunächst mit einem **Array**. Jedes Wartezimmer kann `add(Patient p)` (hinten anstellen), `poll()` (vorne aufrufen) und `ausgeben()`.

Aufgabe 1) Projekt Praxis Website: Wartezimmer-Simulator

Starte das Hauptprogramm nacheinander mit Variante 1, 2 und 3. Notiere Idee und Problem.

Variante	Idee	Was geht schief?
1 Suchen	add: erster freier Platz, poll: erster belegter Platz	Neue rücken vorne in freie Plätze und kommen vor Resi und Bene dran.
2 Nummer	Eine Nummer merkt sich, welcher Platz als nächster dran ist.	Vorne Belegte kommen nie dran, die Nummer läuft übers Array hinaus.
3 Aufrücken	Nach jedem Aufruf rücken alle einen Platz vor.	Reihenfolge stimmt, aber bei jedem Aufruf werden alle verschoben.

Was passiert bei **allen drei** Varianten mit Mia? Sie wird abgewiesen: Das Array ist voll (5 Plätze).

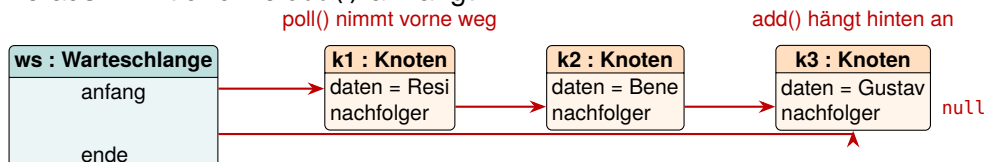
Probleme mit Arrays

Ein Array hat eine **fixe Größe** und belegt schon beim Erzeugen **Speicher** für alle Plätze, auch für leere (`null`). Zum Vergrößern müsste man **ein neues Array** erzeugen und alles **kopieren**. Damit die Reihenfolge stimmt, müssen beim Herausnehmen alle **aufrücken**.

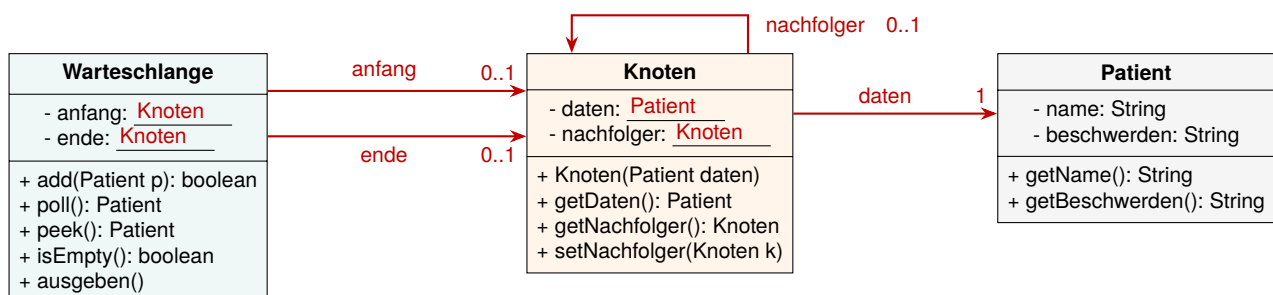
Idee: Jedes Element kennt zusätzlich **seinen Nachfolger**. So entsteht eine **Kette** beliebiger Länge.

Aufgabe 2)

a) Resi, Bene und Gustav warten. Zeichne alle Verweise als Pfeile ein (oder `null`). Markiere, wo `poll()` herausnimmt und wo `add()` anhängt.



b) Ergänze die Datentypen und zeichne die Beziehungen mit Namen und Multiplizitäten ein.

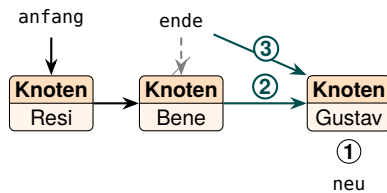


Die Warteschlange (Queue)

- Eine Warteschlange speichert **beliebig viele** Objekte in einer festen **Reihenfolge**.
- Die Warteschlange kennt nur den ersten (`anfang`) und letzten Knoten (`ende`), jeder Knoten seine `daten` und seinen **Nachfolger**. Beim letzten Knoten ist er **null**.
- Eine Beziehung zwischen Objekten **derselben** Klasse heißt **rekursive Beziehung**. Im Code wird sie zu einem Referenzattribut, dessen Typ die **eigene Klasse** ist: `private Knoten nachfolger;`
- `add` hängt **hinten** an, `poll` nimmt **vorne** weg: **FIFO**, First In – First Out.

Aufgabe 3) Projekt Praxis, TODO 3

add(Patient p) braucht drei Schritte. Übersetze jeden Schritt im Bild in eine Zeile Code.



- ① Neuen Knoten für p erzeugen.
- ② Bisher letzter Knoten bekommt neu als Nachfolger.
- ③ ende verweist auf den neuen Knoten.

```
public boolean add(Patient p) {
    Knoten neu = new Knoten(p); // ①
    if (isEmpty()) { // Sonderfall
        anfang = neu;
        ende = neu;
    } else {
        ende.setNachfolger(neu); // ②
        ende = neu; // ③
    }
    return true;
}
```

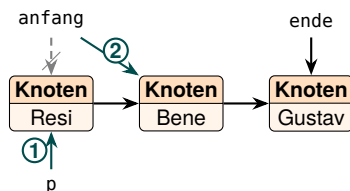
b) Warum braucht man den Sonderfall für die leere Schlange? Was passiert ohne ihn?

ende ist null: ende.setNachfolger(neu) führt zu einer NullPointerException.

c) Übertrage die Methode ins Projekt und teste sie mit new Warteschlange().

Aufgabe 4) Projekt Praxis, TODO 4

poll() holt den vordersten Patienten heraus und gibt ihn zurück.



- ① Die Daten des ersten Knotens in p merken.
- ② anfang rückt auf den Nachfolger weiter.
- ③ War das der letzte Knoten, wird auch ende leer.
- ④ p zurückgeben.

```
public Patient poll() {
    if (isEmpty()) {
        return null; // niemand wartet
    } else {
        Patient p = anfang.getDaten(); // ①
        anfang = anfang.getNachfolger(); // ②
        if (anfang == null) { // ③
            ende = null;
        }
        return p; // ④
    }
}
```

b) Warum muss Schritt 1 vor Schritt 2 stehen?

Nach Schritt 2 verweist nichts mehr auf Resis Knoten: Man käme nicht mehr an ihre Daten.

c) Was geschieht danach mit dem Knoten von Resi?

Kein Verweis zeigt mehr auf ihn: Java räumt ihn automatisch weg (Garbage Collector).

Aufgabe 5) Projekt Praxis

Stelle das Hauptprogramm auf new Warteschlange() um und vergleiche mit Variante 3.

	Variante 3 (Aufrücken)	Warteschlange
Verschiebungen bei poll(), wenn n warten	$n-1$: Der Aufwand wächst mit der Länge.	keine: immer gleich viele Anweisungen
Wie viele können warten?	höchstens 5 (Größe des Arrays)	beliebig viele (nur der Arbeitsspeicher begrenzt)

Es geht auch anders – bespricht mündlich, warum man es so machen kann:

- Die Warteschlange merkt sich **nur anfang**, kein ende. Wie findet add dann den letzten Knoten? Was wird einfacher, was langsamer?
- poll() gibt den **Knoten** statt des Patienten zurück. Warum ist es besser, wenn das Hauptprogramm die Klasse Knoten gar nicht kennen muss?
- Patient bekommt **selbst** ein Attribut nachfolger, die Klasse Knoten entfällt. Was spart man, was verliert man?
- add liefert boolean, obwohl hier immer true herauskommt. Warum ist das trotzdem sinnvoll? (Tipp: LinkedList in der Breitensuche, Klasse 11)